CHAPTER 2

Ruby.new

Many books on programming languages look about the same. They start with chapters on basic types: integers, strings, and so on. Then they look at expressions like `2 + 3`, before moving on to `if` and `while` statements and loops. Then, perhaps around Chapter 7 or 8, they'll start mentioning classes. We find that somewhat tedious.

Instead, when we designed this book, we had a grand plan. We wanted to document the language from the top down, starting with classes and objects and ending with the nitty-gritty syntax details. It seemed like a good idea at the time. After all, most everything in Ruby is an object, so it made sense to talk about objects first.

Or so we thought.

Unfortunately, it turns out to be difficult to describe a language top-down. If you haven't covered strings, `if` statements, assignments, and other details, it's difficult to write examples of classes. Throughout our top-down description, we kept coming across low-level details we needed to cover so that the example code would make sense.

So we came up with another grand plan (they don't call us pragmatic for nothing). We'd still describe Ruby starting at the top. But before we did that, we'd add a short chapter that described all the common language features used in the examples along with the special vocabulary used in Ruby, a mini-tutorial to bootstrap us into the rest of the book. And that mini-tutorial is this chapter.

Ruby Is an Object-Oriented Language

Let's say it again. Ruby is an object-oriented language. In programming terms, an *object* is a thing that combines data with the logic that manipulates that data, and a language is "object-oriented" if it provides language constructs that make it easy to create objects. Typically, object-oriented languages allow their objects to define what their data is, define their functionality, and provide a common syntax to allow other objects to access that functionality.

Unlike other object-oriented languages such as Java, JavaScript, and Python, all Ruby types are objects, and there are no non-object basic types that behave differently. However, many languages claim to be object-oriented, and those languages often have a different interpretation of what object-oriented means and a different terminology for the concepts they employ.

So, before we get too far into the details, let's briefly look at the terms and notations that we'll be using to talk about Ruby.